

Digitalización de Trámites Universitarios con Implementación de BPMN y Almacenamiento Basado en Archivos JSON (junio de 2026)

Cristian Adrian Condori Apaza, Moises Abraham Quispe Mamani, Carrera de Informática, Universidad Mayor de San Andrés, La Paz, Bolivia, ccondoria4@fcpn.edu.bo, mquispem4@fcpn.edu.bo

I. INTRODUCCIÓN

Resumen - Este trabajo presenta el modelado y la implementación de un sistema para la gestión de trámites universitarios basado en Business Process Management (BPM). Se modelaron dos procesos institucionales de la Universidad Mayor de San Andrés (UMSA): la postulación a beca comedor y la habilitación para defensa de grado, utilizando la notación BPMN. A partir de estos modelos se desarrolló un motor de flujo en PHP capaz de interpretar procesos definidos mediante archivos JSON, permitiendo la ejecución, validación de roles y seguimiento de los trámites sin necesidad de programar una lógica específica para cada proceso. Los resultados muestran que el sistema ejecuta correctamente ambos flujos, respetando las condiciones de decisión y registrando la trazabilidad de cada actividad.

Palabras claves: BPM, BPMN, motor de flujo, trámites universitarios, PHP, JSON, automatización de procesos

Digitization of University Administrative Procedures through BPMN Implementation and JSON-based Storage

Abstract - This article presents the modeling and implementation of a system for managing university administrative procedures based on Business Process Management (BPM). Two institutional processes at the Universidad Mayor de San Andrés (UMSA)—the application for a meal plan scholarship and the authorization to defend a thesis—were modeled using BPMN notation. Based on these models, a workflow engine was developed in PHP capable of interpreting processes defined via JSON files, enabling the execution, role validation, and tracking of procedures without the need to program specific logic for each process. The results show that the system correctly executes both workflows, adhering to decision conditions and recording the traceability of each activity.

Keywords: BPM, BPMN, workflow engine, university procedures, PHP, JSON, process automation

En la Universidad Mayor de San Andrés (UMSA), trámites como la postulación a la beca comedor o la habilitación para la defensa de grado requieren la intervención coordinada de distintas unidades trabajo social, comisión de becas, finanzas, tutores, biblioteca, cajas, kardex y dirección de carrera, lo que hace evidente la necesidad de un enfoque estructurado para modelar, automatizar y monitorear dichos procesos.

La disciplina de Gestión de Procesos de Negocio (Business Process Management, BPM), junto con su notación estándar BPMN (Business Process Model and Notation), ofrece un marco adecuado para representar gráficamente este tipo de flujos, identificando actores (lanes), actividades, compuertas de decisión (gateways) y eventos de inicio y fin. A partir de un modelo BPMN correctamente definido, es posible derivar una implementación computacional que ejecute el proceso real, validando en cada paso el rol del usuario y las condiciones de avance.

El presente trabajo documenta el diseño BPMN de dos procesos institucionales de la UMSA y el desarrollo de un motor de flujo de trabajo (workflow engine), implementado en PHP, capaz de ejecutar cualquiera de los dos procesos a partir de su definición almacenada en archivos JSON, sin necesidad de programar una lógica específica para cada trámite.

El artículo se organiza de la siguiente manera: la Sección II presenta el fundamento teórico de BPM, BPMN y los conceptos técnicos empleados en la implementación; la Sección III describe la metodología seguida, desde el modelado BPMN hasta la construcción del motor de flujo; la Sección IV expone los resultados obtenidos; la Sección V presenta las conclusiones; y la Sección VI enumera las referencias consultadas.

Las principales contribuciones de este trabajo son:

- 1) Modelado BPMN de dos trámites institucionales de la UMSA.
- 2) Diseño de un motor de flujo genérico basado en JSON y PHP.
- 3) Implementación de un mecanismo de control de acceso basado en roles.

- 4) Validación experimental mediante dos procesos universitarios.

II. FUNDAMENTO TEÓRICO

A. Gestión de Procesos de Negocio (BPM)

BPM es una disciplina de gestión que combina conocimientos de tecnologías de la información y ciencias administrativas, orientada a mejorar el desempeño de los procesos de negocio de una organización mediante su identificación, modelado, ejecución, monitoreo y optimización continua. A diferencia de los sistemas de información tradicionales, en los cuales la lógica del proceso suele estar codificada de forma rígida dentro del software, el enfoque BPM busca separar la definición del proceso de su motor de ejecución, de modo que los procesos puedan modificarse con mayor agilidad ante cambios organizacionales [2], [3].

B. Notación BPMN

BPMN (Business Process Model and Notation) es el estándar gráfico más utilizado para el modelado de procesos de negocio. Entre sus elementos principales se encuentran los eventos de inicio y fin (representados como círculos), las actividades o tareas (rectángulos de bordes redondeados), las compuertas exclusivas o gateways de tipo XOR (representadas con un rombo que contiene una "X"), utilizadas para modelar puntos de decisión con una única ruta de salida posible, y los carriles o lanes, que permiten asignar cada actividad al actor o rol responsable de ejecutarla dentro de un proceso colaborativo [1].

C. Motor de flujo de trabajo (Workflow Engine)

Un motor de flujo de trabajo es un componente de software encargado de interpretar la definición de un proceso generalmente almacenada en una estructura de datos independiente del código fuente, como una tabla relacional o un documento JSON y de gestionar su ejecución paso a paso, determinando en cada momento cuál es la siguiente actividad, qué rol debe ejecutarla y bajo qué condiciones el proceso avanza hacia una u otra rama. Este enfoque permite que un mismo motor genérico sea capaz de ejecutar distintos procesos de negocio sin necesidad de modificar su código fuente, bastando con cambiar la definición del flujo [2], [3].

D. JSON como formato de definición de procesos

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos, ampliamente utilizado para representar estructuras jerárquicas mediante pares de clave-valor. En el contexto de este proyecto, JSON se emplea como mecanismo de persistencia para describir de forma declarativa tanto la secuencia de actividades de cada proceso como las condiciones de bifurcación en los puntos de decisión, separando así la definición del proceso de la lógica de ejecución del motor [4].

E. Roles y control de acceso basado en roles (RBAC)

El control de acceso basado en roles (Role-Based Access Control, RBAC) es un modelo de seguridad en el cual los permisos para ejecutar determinadas acciones se asignan a roles y no directamente a usuarios individuales, de modo que cada usuario hereda los permisos del rol que tiene asignado. En el

sistema desarrollado, este modelo se utiliza para garantizar que cada paso del proceso BPMN únicamente pueda ser ejecutado por el actor correspondiente (por ejemplo, que la actividad "Recepcion_TS" solo pueda ser realizada por un usuario con rol "TRABAJO_SOCIAL") [5].

III. METODOLOGÍA

El desarrollo del proyecto se llevó a cabo en tres etapas: (1) modelado BPMN de los dos procesos institucionales, (2) diseño de la estructura de datos JSON que representa dichos procesos de forma genérica, y (3) implementación del motor de flujo en PHP y de los módulos de interfaz correspondientes a cada paso del proceso.

A. Modelado BPMN de los procesos

Se modelaron dos procesos institucionales de la UMSA utilizando la notación BPMN, cada uno representado mediante carriles (lanes) correspondientes a los distintos actores involucrados.

El primer proceso, denominado "Postulación Beca Comedor", involucra cuatro roles: Estudiante, Trabajo Social, Comisión de Becas y Finanzas. El flujo inicia cuando el estudiante completa un formulario; Trabajo Social recibe la solicitud y verifica la documentación mediante una compuerta de decisión (¿Es verificado?); si no es verificado, el flujo retorna al estudiante para corregir el formulario; si es verificado, se genera un informe que pasa a la Comisión de Becas, la cual evalúa si la solicitud está aprobada (¿Está aprobado?); en caso de no estarlo, se notifica al estudiante el rechazo (Reprobado); si es aprobada, Finanzas inscribe al estudiante en el beneficio y finalmente se emite una resolución final que cierra el proceso.

Fig. 1. Diagrama BPMN Beca Comedor

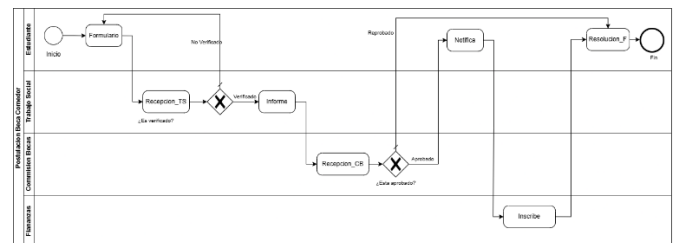


Tabla. 1. Detalles de procesos del flujo F1

Flujo	Proceso	Sig. Proceso	Tipo	Rol	Pantalla
F1	P1	P2	I	ESTUDIANTE	formulario.inc.php
F1	P2	P3	P	TRABAJO_SOCIAL	recepcion_ts.inc.php
F1	P3	null	Q	TRABAJO_SOCIAL	verifica.inc.php
F1	P4	P5	P	TRABAJO_SOCIAL	informe.inc.php
F1	P5	P6	P	COMISION_BECAS	recepcion_cb.inc.php
F1	P6	null	Q	COMISION_BECAS	aprobado.inc.php
F1	P7	P8	P	ESTUDIANTE	notifica.inc.php
F1	P8	P9	P	FINANZAS	inscribe.inc.php
F1	P9	null	F	ESTUDIANTE	resolucion_f.inc.php

El segundo proceso, "Habilitación Para Defensa De Grado", involucra seis roles: Estudiante, Tutor, Biblioteca, Cajas, Kardex y Director. El estudiante presenta su documento, el cual es evaluado por el Tutor (¿Es aceptado?); si es rechazado, el

documento es devuelto al estudiante; si es aceptado, Biblioteca verifica si el estudiante adeuda libros (¿Debe libros?), en cuyo caso se solicita la devolución correspondiente; si no adeuda libros, Cajas verifica si el estudiante tiene deudas pendientes (¿Tiene deudas?), solicitando el pago en caso afirmativo; una vez resueltas las posibles deudas, Kardex redacta la documentación de habilitación, el Director firma la resolución y finalmente se notifica al estudiante la conclusión del trámite.

Fig. 2. Diagrama BPMN Defensa de Grado

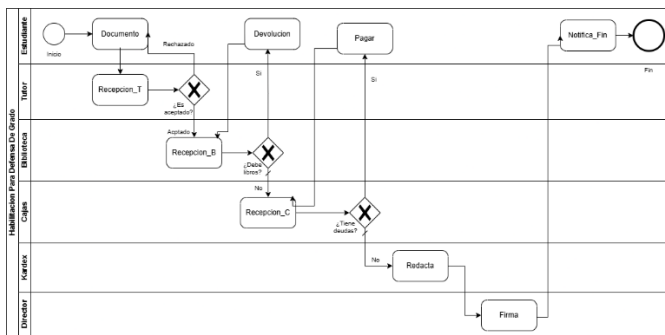


Tabla. 2. Detalles de procesos del flujo F2

Flujo	Proceso	Sig. Proceso	Tipo	Rol	Pantalla
F2	P1	P2	I	ESTUDIANTE	documento.inc.php
F2	P2	P3	P	TUTOR	recepcion_t.inc.php
F2	P3	null	Q	TUTOR	evalua.inc.php
F2	P4	P5	P	BIBLIOTECA	recepcion_b.inc.php
F2	P5	null	Q	BIBLIOTECA	verifica_b.inc.php
F2	P6	P4	P	ESTUDIANTE	devolucion.inc.php
F2	P7	P8	P	CAJAS	recepcion_c.inc.php
F2	P8	null	Q	CAJAS	verifica_c.inc.php
F2	P9	P7	P	ESTUDIANTE	pagar.inc.php
F2	P10	P11	P	KARDEX	redacta.inc.php
F2	P11	P12	P	DIRECTOR	firma.inc.php
F2	P12	null	F	ESTUDIANTE	notifica_fin.inc.php

B. Estructura de datos JSON del motor de flujo

Para que un único motor pudiera ejecutar ambos procesos sin necesidad de codificar reglas específicas para cada uno, se diseñaron cuatro archivos JSON ubicados en el directorio de datos del sistema, descritos a continuación.

El archivo flujo_proceso.json define, para cada proceso (identificado como "F1" para Beca Comedor y "F2" para Habilitación de Grado), la secuencia ordenada de pasos (P1, P2, P3, ...), el paso siguiente por defecto, el tipo de actividad (I = inicio, P = tarea de proceso, Q = compuerta de decisión, F = fin), el rol autorizado para ejecutar dicho paso y la pantalla o módulo PHP asociado. Un fragmento representativo de esta estructura es el siguiente:

```
[
  { "flujo": "F1", "proceso": "P1",
    "procesosiguiente": "P2", "tipo": "I", "rol":
    "ESTUDIANTE", "pantalla": "formulario" },
  { "flujo": "F1", "proceso": "P3",
    "procesosiguiente": null, "tipo": "Q", "rol":
    "TRABAJO_SOCIAL", "pantalla": "verifica" },
```

```
{ "flujo": "F2", "proceso": "P1",
  "procesosiguiente": "P2", "tipo": "I", "rol":
  "ESTUDIANTE", "pantalla": "documento" },
  { "flujo": "F2", "proceso": "P2",
    "procesosiguiente": "P3", "tipo": "P", "rol":
    "TUTOR", "pantalla": "recepcion_t" }
]
```

El archivo flujo_condicion.json complementa al anterior, definiendo —únicamente para los pasos de tipo compuerta (Q)— hacia qué proceso debe avanzar el flujo según la condición evaluada (SI o NO). Por ejemplo:

```
[
  { "flujo": "F1", "proceso": "P3", "condicion": "SI",
    "procesosiguiente": "P4" },
  { "flujo": "F1", "proceso": "P3", "condicion": "NO",
    "procesosiguiente": "P1" },
  { "flujo": "F2", "proceso": "P3", "condicion": "SI",
    "procesosiguiente": "P4" },
  { "flujo": "F2", "proceso": "P3", "condicion": "NO",
    "procesosiguiente": "P1" }
]
```

El archivo usuarios.json almacena las credenciales y el rol de cada usuario del sistema (estudiante, trabajo social, comisión de becas, finanzas, tutor, biblioteca, cajas, kardex y director), información empleada por el motor para validar que únicamente el rol autorizado pueda ejecutar cada paso del proceso. Finalmente, el archivo seguimiento.json funciona como una bitácora de ejecución: por cada trámite (identificado mediante un número de trámite, nrotramite), se registra el flujo al que pertenece, el proceso ejecutado, el usuario y rol que lo realizó, así como la fecha y hora de inicio (fechaini) y de finalización (fechafin) de dicho paso, permitiendo reconstruir la trazabilidad completa del trámite.

C. Implementación del motor de flujo en PHP

Sobre la base de la estructura de datos descrita, se desarrolló un motor de flujo en PHP (motor.php) encargado de leer los archivos JSON, determinar el estado actual de un trámite, validar el rol del usuario autenticado frente al rol requerido por el siguiente paso, invocar el módulo correspondiente a la pantalla asociada y, una vez completada la actividad, registrar el evento correspondiente en seguimiento.json y calcular el proceso siguiente ya sea de forma directa (campo procesosiguiente de flujo_proceso.json) o evaluando la condición correspondiente cuando el paso actual es una compuerta de decisión (flujo_condicion.json).

Alrededor de este motor genérico se desarrollaron los módulos de interfaz correspondientes a cada actividad de ambos procesos (formulario.inc.php, recepcion_ts.inc.php, verifica.inc.php, informe.inc.php, recepcion_cb.inc.php, aprobado.inc.php, notifica.inc.php, inscribe.inc.php y resolucion_f.inc.php para el proceso de Beca Comedor; documento.inc.php, recepcion_t.inc.php, evalua.inc.php, recepcion_b.inc.php, verifica_b.inc.php, devolucion.inc.php, recepcion_c.inc.php, verifica_c.inc.php, pagar.inc.php, redacta.inc.php, firma.inc.php y notifica_fin.inc.php para el proceso de Habilitación de Grado), además de módulos transversales como login.php, logout.php, index.php, nuevo_tramite.php y bandeja.php, este último encargado de mostrar a cada usuario los trámites pendientes de atención según su rol.

El sistema fue desplegado sobre un entorno de servidor local XAMPP [7], alojando el proyecto bajo el directorio htdocs (tramites_umsa), el motor de flujo y los módulos de interfaz fueron desarrollados utilizando el lenguaje PHP [6], con los cuatro archivos JSON ubicados en una subcarpeta dedicada (datos), separando así claramente la capa de definición de procesos de la capa de presentación e interfaz del sistema.

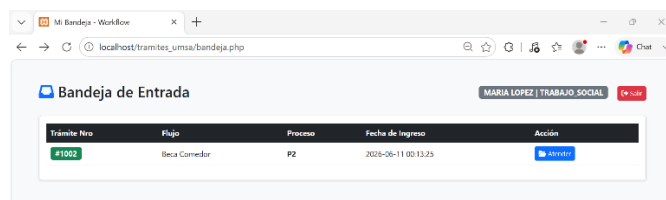
IV. RESULTADOS

Como resultado del desarrollo descrito, se obtuvo un sistema web funcional capaz de ejecutar de manera independiente y simultánea los dos procesos modelados —Postulación Beca Comedor y Habilitación Para Defensa de Grado—, a partir de un mismo motor de flujo, sin que fuera necesario programar lógica adicional específica para cada trámite más allá de los módulos de interfaz de cada paso.

La validación de roles mediante el archivo usuarios.json permitió comprobar que cada actividad del proceso únicamente pudo ser ejecutada por el actor correspondiente; por ejemplo, el paso "recepcion_ts" del proceso de Beca Comedor solo resultó accesible para el usuario con rol TRABAJO_SOCIAL, mientras que el paso "firma" del proceso de Habilitación de Grado solo resultó accesible para el usuario con rol DIRECTOR, replicando fielmente la asignación de carriles (lanes) definida en los diagramas BPMN originales.

La Fig. 3 muestra un ejemplo de la bandeja de tareas para un usuario con rol TRABAJO_SOCIAL. En ella se listan los trámites que han llegado al paso recepcion_ts o verifica (actividades asignadas a dicho rol dentro del proceso F1) Esta interfaz permite a los operadores institucionales gestionar eficientemente las solicitudes entrantes sin necesidad de conocer la secuencia completa del proceso, ya que el motor determina automáticamente el siguiente paso y la pantalla asociada.

Fig. 3. Interfaz de la Bandeja de entrada de Trabajo Social



El archivo seguimiento.json evidenció el registro correcto de la trazabilidad de un trámite de prueba (nrotramite 1001) del proceso F2, mostrando la secuencia cronológica de pasos ejecutados por distintos usuarios (CRISTIAN en el rol ESTUDIANTE y TUTOR1 en el rol TUTOR), con marcas de fecha y hora de inicio y fin de cada actividad, lo cual confirma que el motor es capaz de mantener un historial auditable y consistente del avance de cada solicitud.

Para visualizar cómo se organiza la información capturada por el sistema, se puede observar el siguiente formato:

```
{ "nrotramite": 1001,
  "flujo": "F2",
  "proceso": "P1",
  "usuario": "CRISTIAN",
  "rol": "ESTUDIANTE",
```

```
"fechaini": "2026-06-10 23:59:42",
"fechafin": "2026-06-10 23:59:52"},

{ "nrotramite": 1001,
  "flujo": "F2",
  "proceso": "P2",
  "usuario": "TUTOR1",
  "rol": "TUTOR",
  "fechaini": "2026-06-10 23:59:52",
  "fechafin": "2026-06-11 00:00:09"},
```

Asimismo, las compuertas de decisión definidas en flujo_condicion.json reprodujeron correctamente el comportamiento de bifurcación modelado en BPMN: en el proceso F1, una condición "NO" en el paso de verificación (P3) retornó el flujo al estudiante (P1) para la corrección del formulario, mientras que una condición "SI" permitió el avance hacia la generación del informe (P4), replicando el comportamiento de la compuerta "¿Es verificado?" del diagrama original.

Tabla 3. Resumen comparativo de los procesos modelados

Características	F1: BECA COMEDOR	F2. Defensa de Grado
Número de roles	4	6
Número de pasos	9	12
Compuertas (Q)	2	3
Módulos PHP	9	12

Comparación del número de roles, pasos, compuertas de decisión y módulos PHP entre los dos procesos modelados.

En conjunto, los resultados confirman que el enfoque de separar la definición del proceso (BPMN traducido a JSON) de su motor de ejecución (PHP) resulta viable y escalable: la incorporación de un tercer proceso institucional requeriría únicamente añadir sus respectivas entradas en flujo_proceso.json y flujo_condicion.json, junto con los módulos de interfaz de cada paso nuevo, sin necesidad de modificar el motor genérico ya implementado.

El código fuente completo del motor de flujo y los módulos de interfaz está disponible en un repositorio público para su replicabilidad.

(<https://github.com/cristiancondoriapaza/SistemaDigitalizacion>).

V. CONCLUSIÓN

El presente trabajo demostró la viabilidad de modelar procesos administrativos universitarios mediante la notación BPMN y de traducir dicho modelo a una estructura de datos JSON ejecutable por un motor de flujo genérico desarrollado en PHP. La separación entre la definición del proceso y la lógica de ejecución permitió que un mismo motor gestionara dos trámites institucionales distintos —Postulación a Beca Comedor y Habilitación para Defensa de Grado—, cada uno con su propio conjunto de roles, actividades y compuertas de decisión, sin necesidad de duplicar o reescribir la lógica central del sistema.

La incorporación de un esquema de control de acceso basado en roles garantizó que cada actividad del proceso fuera ejecutada únicamente por el actor institucional correspondiente, replicando fielmente la asignación de carriles definida en los

diagramas BPMN, mientras que el registro de seguimiento (seguimiento.json) aportó trazabilidad y auditoría sobre cada trámite gestionado por el sistema.

REFERENCIAS

- [1] Object Management Group, "Business Process Model and Notation (BPMN), Version 2.0," OMG Document Number formal/2011-01-03, 2011. [En línea]. Disponible en: <https://www.omg.org/spec/BPMN/2.0/>
- [2] M. Weske, Business Process Management: Concepts, Languages, Architectures, 3.^a ed. Berlín: Springer-Verlag, 2019.
- [3] M. Dumas, M. La Rosa, J. Mendling y H. A. Reijers, Fundamentals of Business Process Management, 2.^a ed. Berlín: Springer-Verlag, 2018.
- [4] ECMA International, "The JSON Data Interchange Syntax," Standard ECMA-404, 2.^a ed., dic. 2017. [En línea]. Disponible en: <https://www.ecma-international.org/publications-and-standards/standards/ecma-404/>
- [5] D. F. Ferraiolo and D. R. Kuhn, "Role-Based Access Control," in Proc. 15th National Computer Security Conference, 1992, pp. 554–563.
- [6] The PHP Group, "PHP: Hypertext Preprocessor — Manual," PHP Documentation. [En línea]. Disponible en: <https://www.php.net/manual/es/>
- [7] Apache Friends, "XAMPP Documentation," Apache Friends. [En línea]. Disponible en: <https://www.apachefriends.org/es/docs/>